

Architettura degli Elaboratori



Prof. Ivan Lanese

The diagram illustrates the internal architecture of the Intel Pentium Pro processor, showing the flow of data and instructions between various components. The main components and their interconnections are as follows:

- External Buses:**
 - Data Bus (64-bit):** Connects to the Bus Unit and the Dual-Access Data Cache.
 - Address Bus (32-bit):** Connects to the Bus Unit and the Dual-Access Data Cache.
 - Control Bus:** Connects to the Bus Unit.
- Bus Unit:** Acts as the central interface for external communication.
- Internal Processor Components:**
 - Branch Target Buffer (BTB):** Provides branch targets to the Instruction Cache.
 - Instruction Cache (8K, 2-way):** Stores instructions for the processor.
 - Prefetch Buffers:** Buffer instructions for the Instruction Decode stage.
 - Instruction Decode:** Decodes instructions and sends control signals to the Control Unit and Microcode ROM.
 - Control Unit:** Coordinates the execution of instructions and sends control signals to the ALU, Barrel Shifter, and FP Register File.
 - Microcode ROM:** Provides microcode for complex instructions.
 - Address Generate U-pipe and V-pipe:** Generate addresses for the Integer Register File and FP Register File.
 - Integer Register File:** Contains 32 general-purpose registers.
 - ALU U-pipe and V-pipe:** Perform integer arithmetic and logic operations.
 - Barrel Shifter:** Performs barrel shifting operations.
 - FP Register File:** Contains 80 floating-point registers.
 - Add, Divide, Multiply:** Perform floating-point operations.
 - Dual-Access Data Cache (8K, 2-way):** Provides fast access to data for both the processor and the system bus.
 - TLB (Translation Lookaside Buffer):** Stores recently used virtual-to-physical address translations.
- Data Flow:**
 - Instructions flow from the Instruction Cache through the Prefetch Buffers and Instruction Decode to the Control Unit.
 - Data flows from the Dual-Access Data Cache to the ALU, Barrel Shifter, and FP Register File.
 - Control signals flow from the Control Unit to the ALU, Barrel Shifter, and FP Register File.

The diagram shows a 1-bit ALU with the following components and connections:

- Inputs:** `instruction`, `inM`, `reset`, and `pc` (Program Counter output).
- Internal Components:**
 - decode:** Receives `instruction` and outputs a control signal `c`.
 - Mux (1):** Selects between `inM` and the `ALU output` based on control signal `c`. Its output is `A`.
 - A:** A 1-bit register that stores the value of `A`. It has a clock input `c` and a reset input.
 - Mux (2):** Selects between `A` and `inM` based on control signal `c`. Its output is `M`.
 - PC:** A 1-bit register that stores the value of `pc`. It has a clock input `c` and a reset input.
 - ALU:** Performs operations on `A` and `M` based on control signal `c`. Its output is `outM`.
 - Feedback Loop:** The `ALU output` is fed back to the `Mux (1)` and the `PC`.
- Outputs:** `outM`, `writeM` (control signal `c`), `addressM` (control signal `c`), and `pc`.

Introduzione

Perchè studiare l'Architettura degli Elaboratori

Applicazioni software: Conoscete già una grande quantità di applicazioni software

Svilupperete nuove applicazioni software: Corsi come "Programmazione", "Linguaggi", "Ingegneria del Software", ... vi forniranno conoscenze e strumenti per sviluppare nuove applicazioni software

Ma cosa permette a tali applicazioni di funzionare?
Il corso di Architettura degli Elaboratori cerca di dare risposta a tale domanda

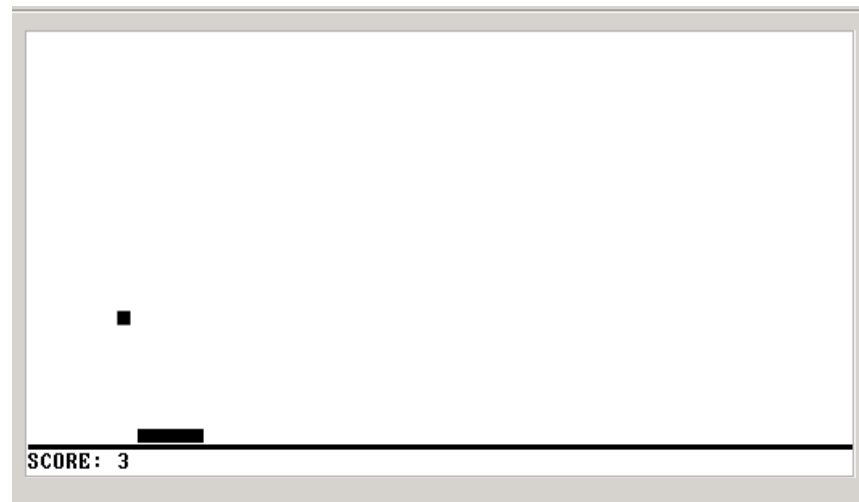
Un caso di studio



Pong, 1985



Pong, 2011



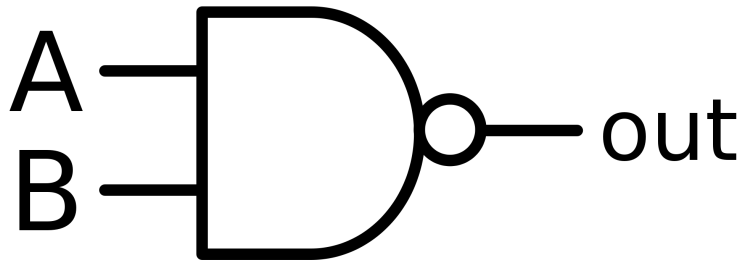
Il nostro "Pong"

Un caso di studio

Non solo studieremo ..

.. ma **REALIZZEREMO** in tutti i suoi minimi dettagli un elaboratore che permette di eseguire il nostro "Pong"

Cosa useremo? Come unico "mattoncino" useremo la porta logica NAND

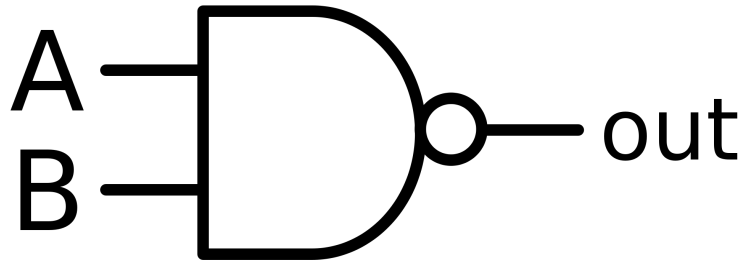


A	B	out
0	0	1
0	1	1
1	0	1
1	1	0

(useremo anche fili per collegare gli ingressi e le uscite dei "mattoncini", un display 256x512 pixel b/w e una tastiera)



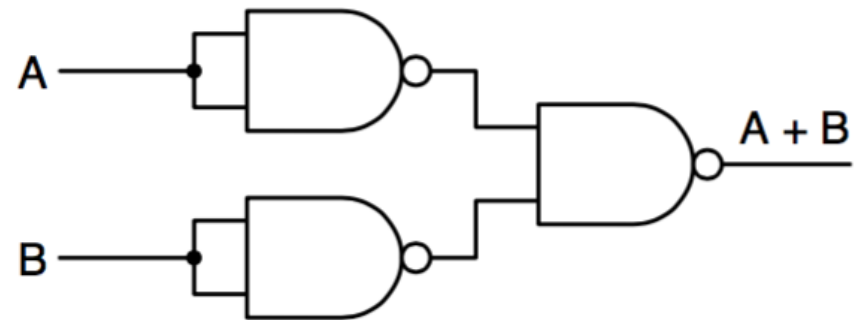
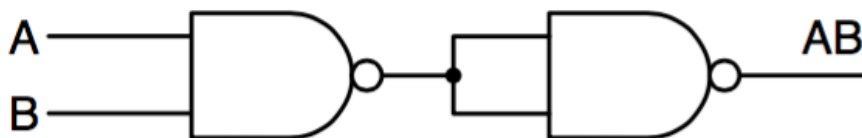
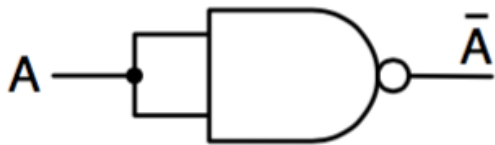
Perchè usare la porta logica NAND?



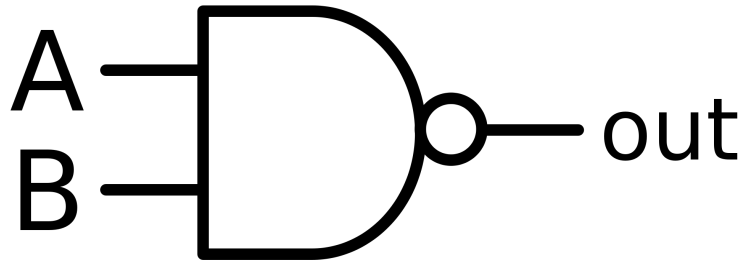
A	B	out
0	0	1
0	1	1
1	0	1
1	1	0

E' una porta logica **UNIVERSALE**:

combinando porte NAND si possono realizzare tutte le funzioni logiche "booleane" (= funzioni a due valori 0,1)



Perchè usare la porta logica NAND?



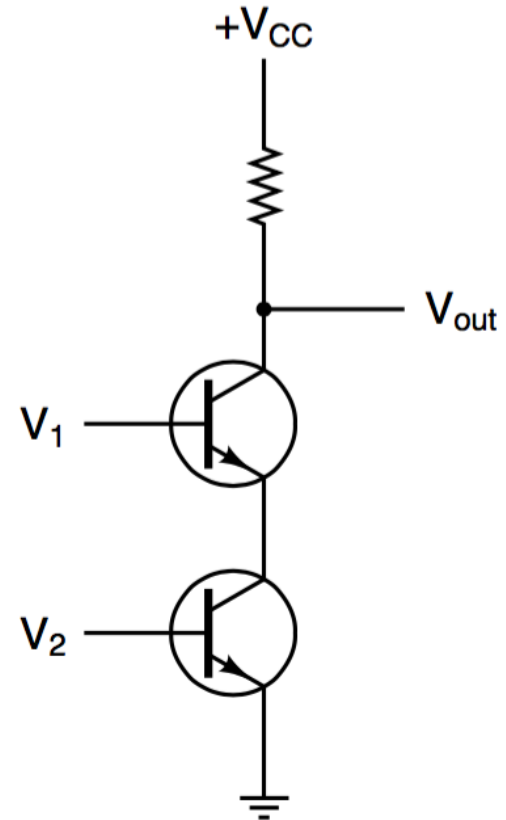
A	B	out
0	0	1
0	1	1
1	0	1
1	1	0

Inoltre è facile da realizzare usando
"transistor"

(0 = no tensione ,
1 = tensione)

(transistor:

- senza tensione in ingresso è una resistenza infinita
- con tensione in ingresso è un conduttore ideale)



Ma quanto è difficile?

Complessità: Realizzare il nostro "Pong" usando solo porte NAND è estremamente complesso!

Come affrontare tale complessità?

- useremo il principio di "astrazione / implementazione"
- tale principio è uno dei concetti base dell'Informatica ..
- .. ma di che si tratta?

Astrazione / implementazione: si crea un "mattoncino" (implementazione) e poi lo si usa dimenticandosi (astrazione) di come è stato costruito

Uso del principio di astrazione/implementazione

Astrazione/implementazione fondamentale in ogni campo

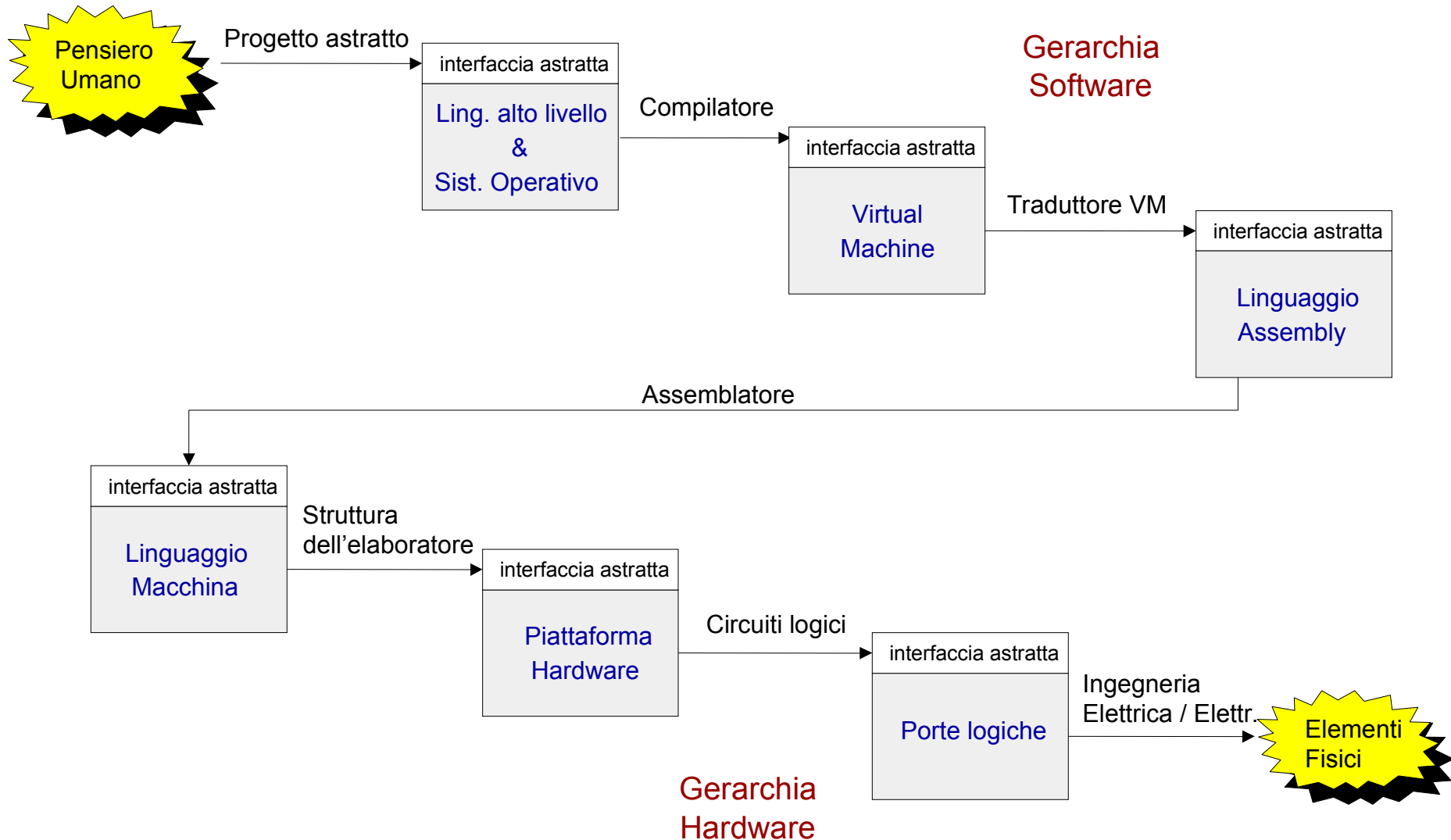
- posso usare la tv senza sapere come è fatta internamente
- posso scrivere un programma senza sapere come sono eseguite le istruzioni
- posso chiamare una funzione senza sapere il suo codice

Mi basta conoscere le loro **interfacce**

Come realizzeremo il nostro "Pong"?

- implementando "Pong" in un linguaggio per una "virtual machine"
- tradurremo il linguaggio per "virtual machine" in linguaggio assembly
- tradurremo il linguaggio assembly in linguaggio macchina
- eseguiremo il linguaggio macchina usando un elaboratore elettronico
- realizzando l'elaboratore usando porte logiche NAND

Astrazione / Implementazione che useremo per il nostro “Pong”



Principio di astrazione/implementazione

Astrazione: Si presenta la soluzione ad un problema concentrandosi solo su alcuni aspetti "rilevanti"
- ad esempio, come ottenere la soluzione componendo soluzioni di problemi più semplici

Implementazione: Si realizza la soluzione aggiungendo gli aspetti astratti nella prima fase
- ad esempio, si mostra come si possono risolvere i problemi più semplici

Gli elaboratori digitali sono macchine multilivello

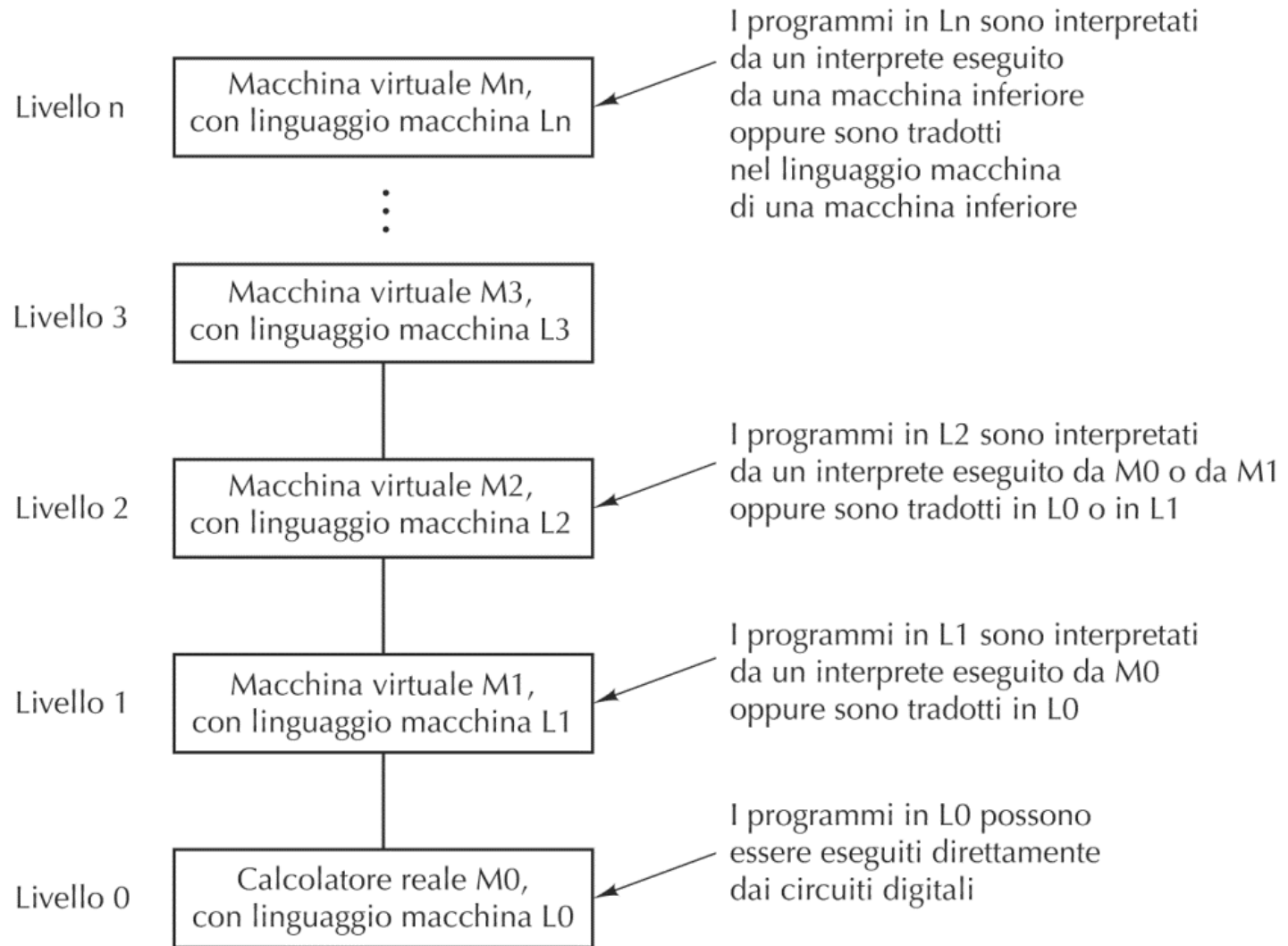


Figura 1.1 Macchina multilivello.

Tipico elaboratore a 6 livelli

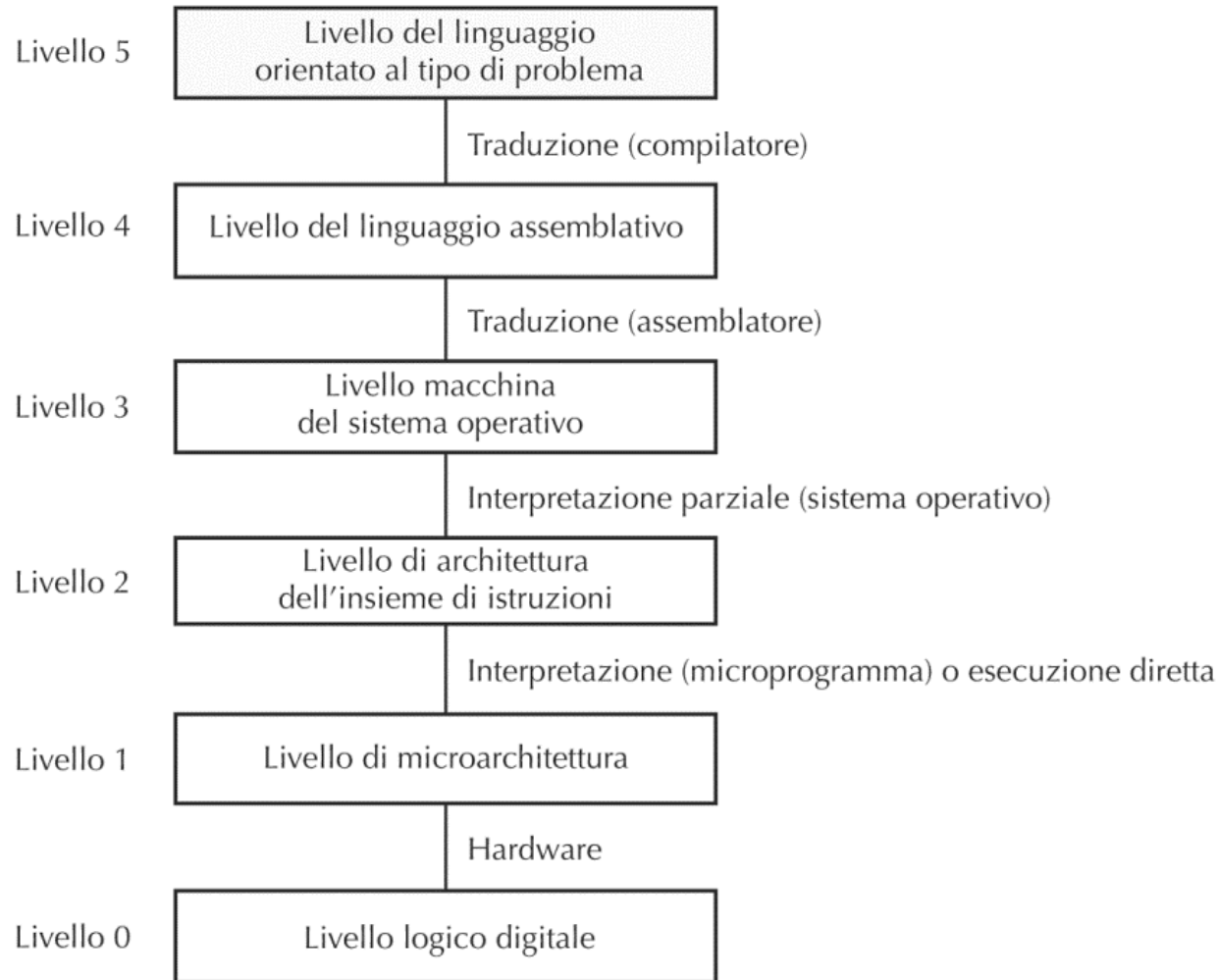
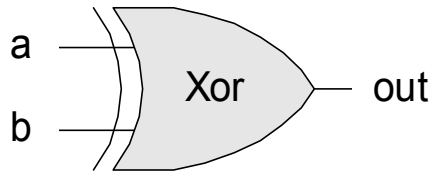


Figura 1.2 Computer a sei livelli. Sotto ciascun livello è indicato il metodo di supporto (oltre al nome del programma corrispondente).

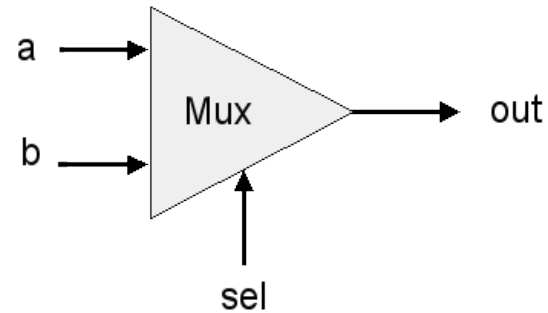
Livello 0: livello logico digitale

Porte logiche:



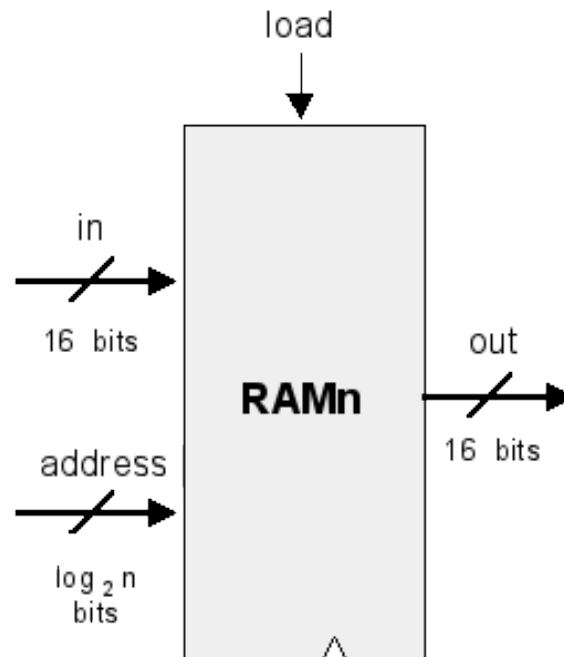
a	b	out
0	0	0
0	1	1
1	0	1
1	1	0

Circuiti combinatori:

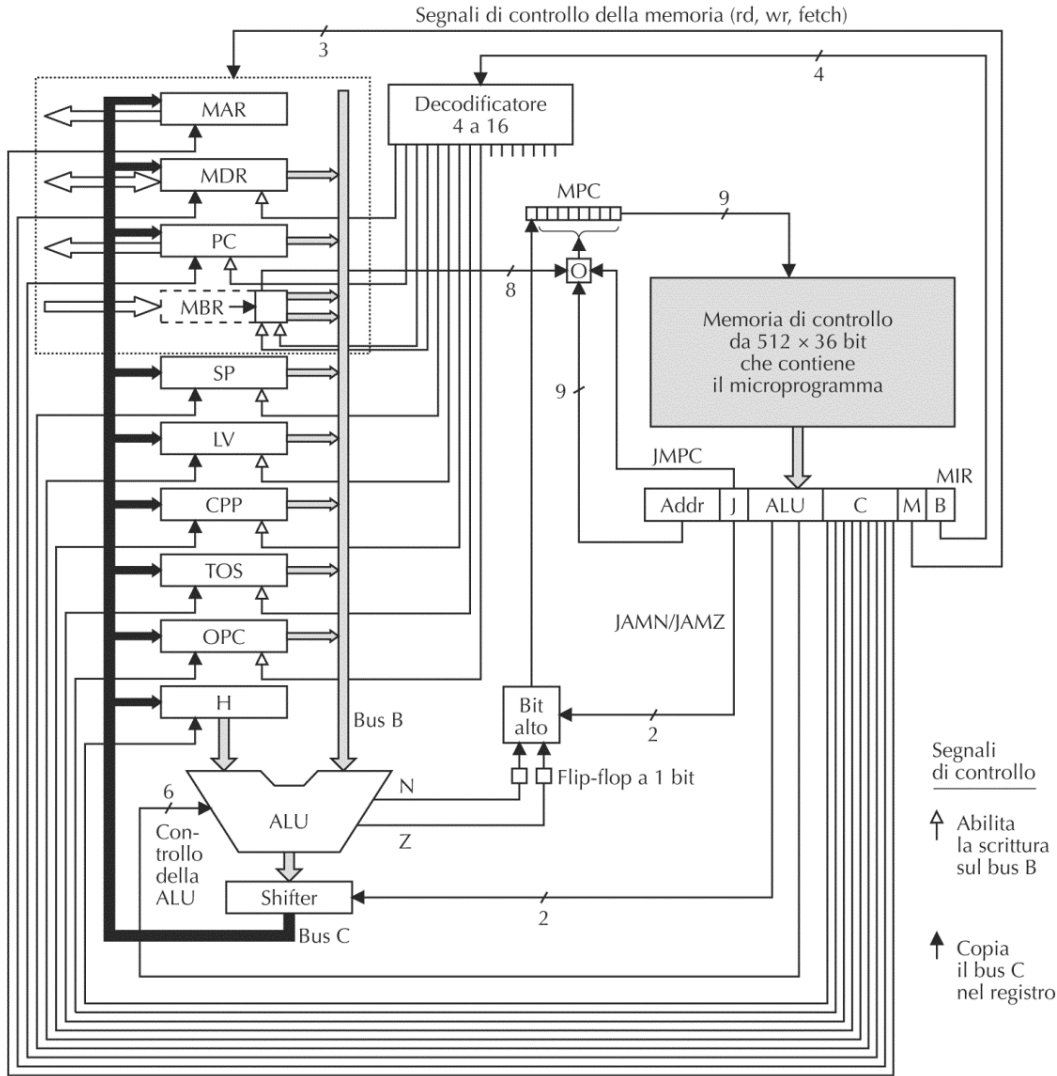


a	b	sel	out
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Circuiti sequenziali:



Livello 1: livello di microarchitettura



Microarchitettura:

governa il flusso
dei dati fra i vari
componenti del
livello logico
digitale

Hardware o Software?:

in alcuni elaboratori è software (vedi esempio), in altri è realizzato in hardware (nostro "Pong")

Figura 4.6 Diagramma a blocchi del nostro esempio di microarchitettura di Mic-I.

Livello 2: livello ISA (Instruction Set Architecture)

Istruzioni macchina:

insieme di istruzioni eseguibili dalla microarchitettura

Livello solitamente considerato come "interfaccia" fra hardware e software

31	28	27					16	15					8	7					0	Tipo di istruzione								
Cond	0	0	I	Opcode			S	Rn		Rd		Operand2						Elaborazione dati / Traferimento di PSR										
Cond	0	0	0	0	0	0	0	A	S	Rd		Rn		RS	1		0	0	1	Rm		Moltiplicazione						
Cond	0	0	0	0	1	U		A	S	RdHi		RdLo		RS	1		0	0	1	Rm		Moltiplicazione Long						
Cond	0	0	0	1	0	B		0		0	Rn		Rd		0		0	0	0	1		0	0	1	Rm	Swap		
Cond	0	1	I	P	U	B	W	L	Rn		Rd		Offset						Load/Store Byte/Word									
Cond	1	0	0	P		U	S	W	L	Rn		Register List						Load/Store Multipla										
Cond	0	0	0	P		U	1	W	L	Rn		Rd		Offset1		1	S	H	1	Offset2		Trasferimento di mezza parola: spiazzamento immediato						
Cond	0	0	0	P		U	0	W	L	Rn		Rd		0		0	0	0	1	S	H	1	Rm		Trasferimento di mezza parola: spiazzamento relativo a registro			
Cond	1	0	1	L		Offset												Salto										
Cond	0	0	0	1	0			0	1	0	1		1	1	1	1		1	1	0		0	0	1	Rn		Scambio di salto (branch exchange)	
Cond	1	1	0	P		U	N	W	L	Rn		CRd		CPNum		Offset						Trasferimento dati del coprocessore						
Cond	1	1	1	0	Op1					CRn		CRd		CPNum		Op2		0			CRm		Operazione su dati del coprocessore					
Cond	1	1	1	0	Op1			L		CRn		Rd		CPNum		Op2		1			CRm		Trasferimento registri del coprocessore					
Cond	1	1	1	1	SWI Number												Interrupt software											

Figura 5.14 Formati delle istruzioni a 32 bit di ARM.

Livello 3-4: Sistema Operativo - Ling. Assemblativo

Livelli "ibridi": non sono così rigidamente separati come i livelli precedenti

Sistema Operativo:
fornisce servizi per gestione risorse (memoria, I/O, ...) ed esecuzione processi

Linguaggio Assemblativo:
permette di programmare i livelli sottostanti (nessuno programma in linguaggio macchina!)

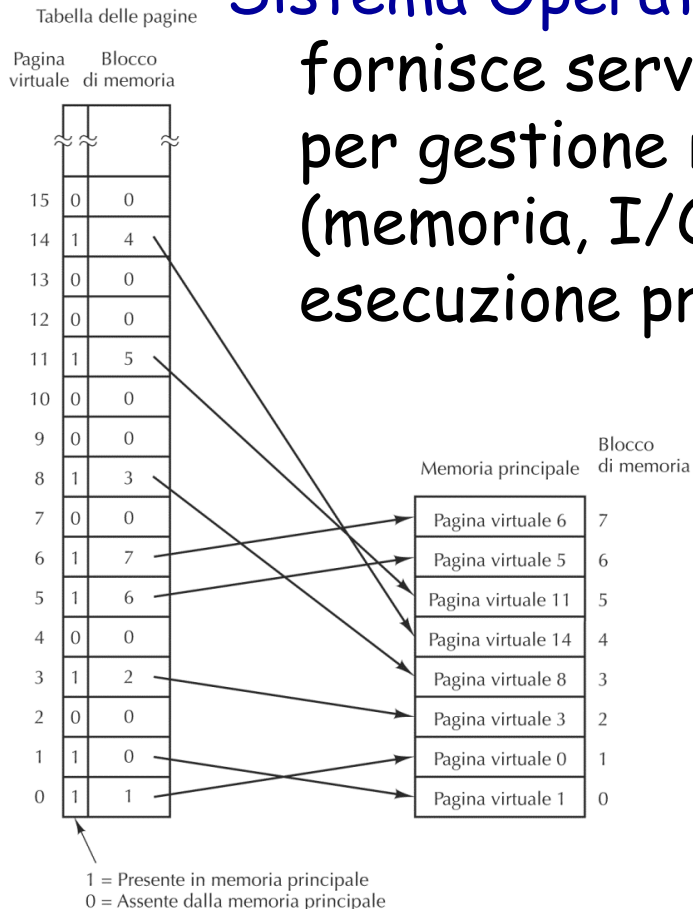


Figura 6.5 Assegnamento di 16 pagine virtuali in una memoria principale composta di otto blocchi.

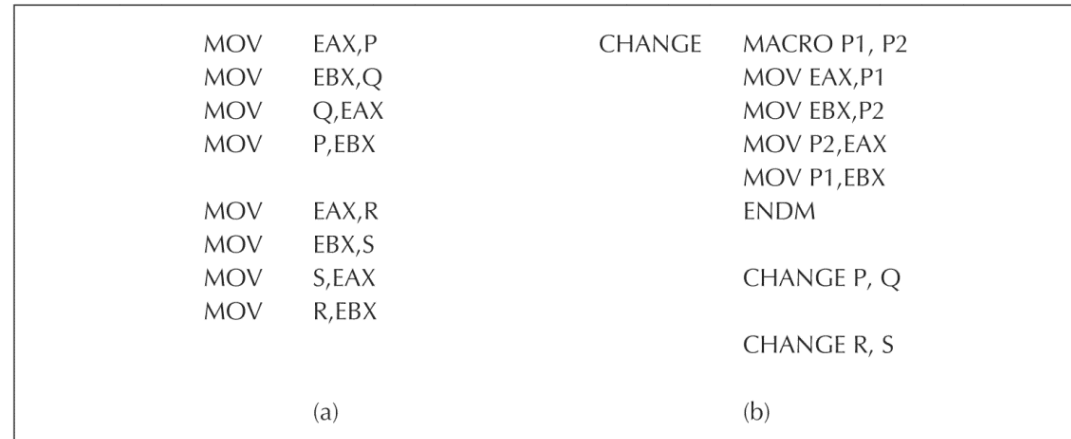
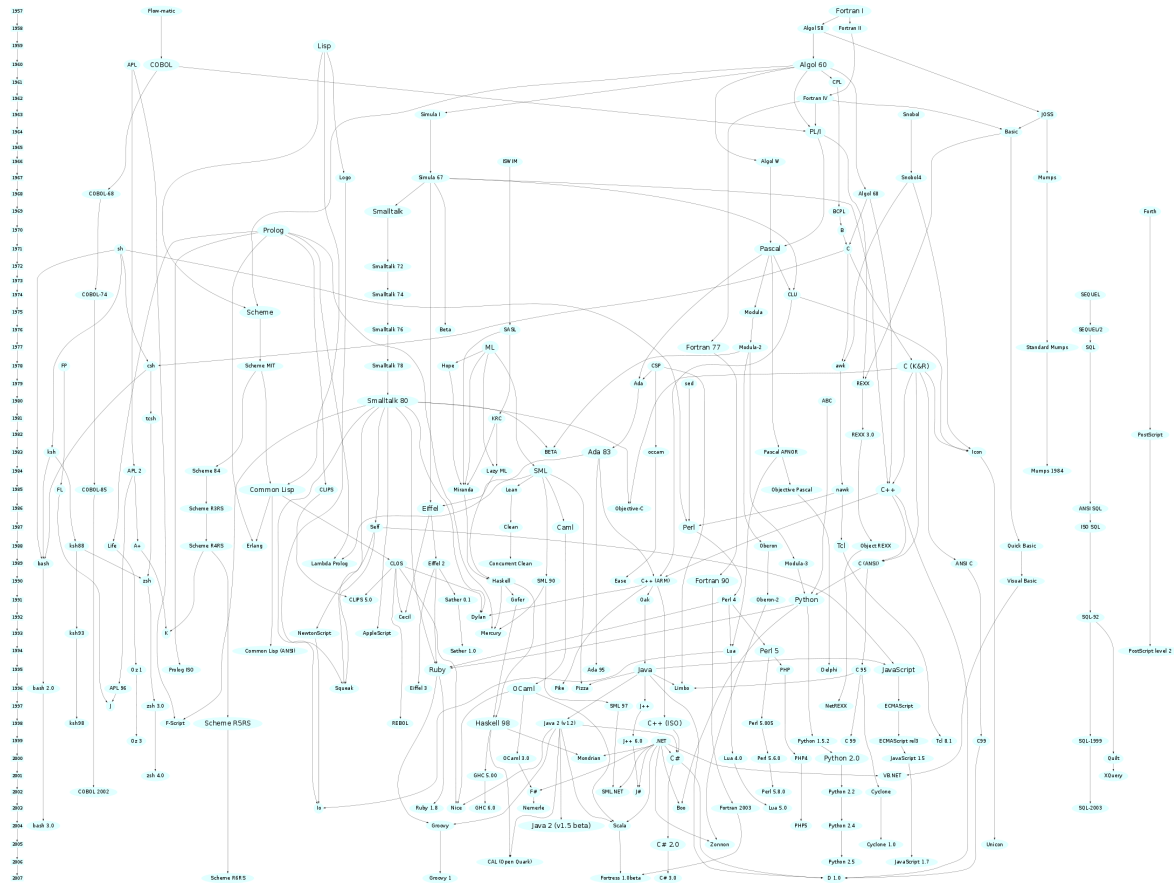


Figura 7.5 Sequenze d'istruzioni quasi identiche. Senza (a) e con (b) macro.

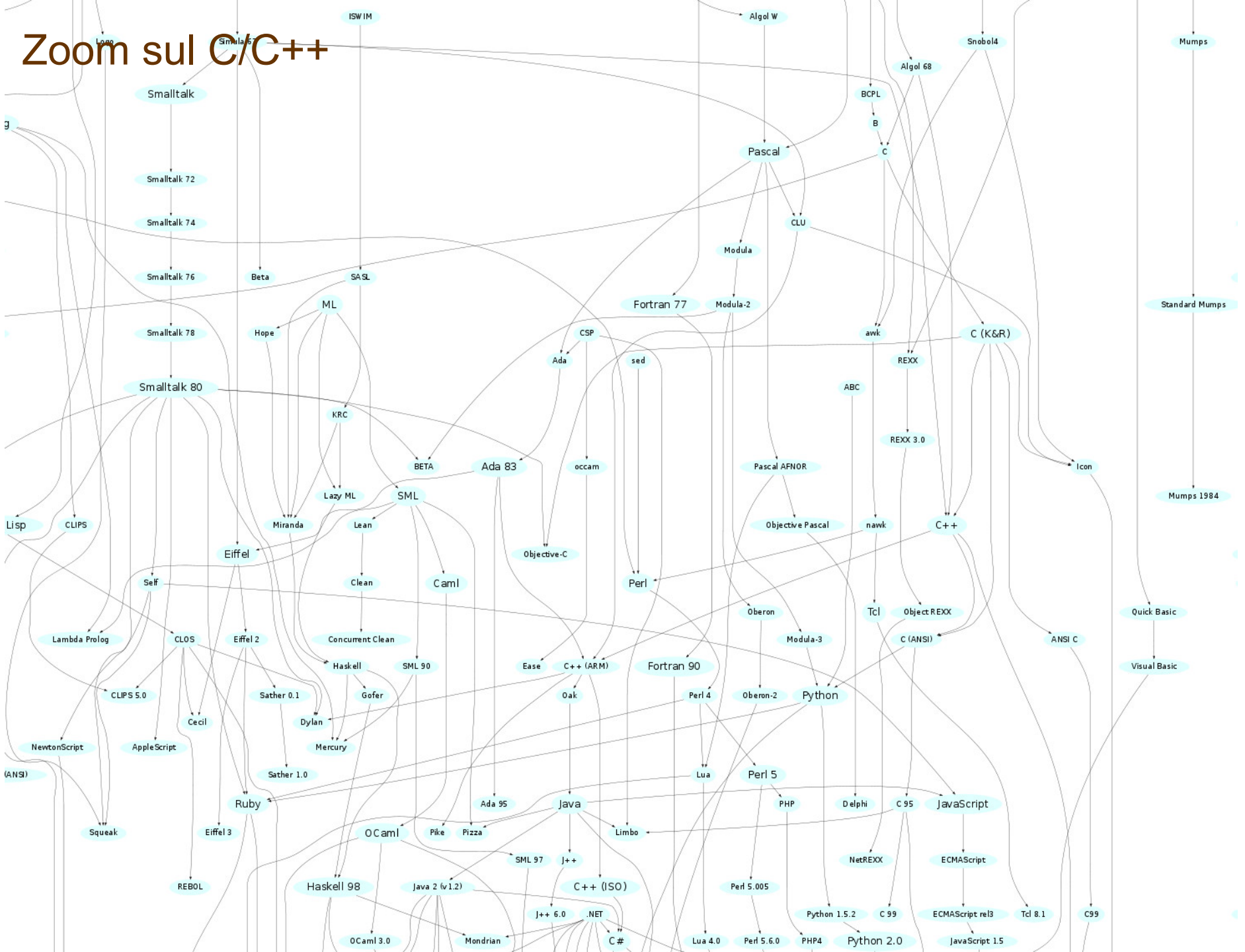
Livello 5: livello del linguaggio orientato al problema

Linguaggi di programmazione ad alto livello:

esiste una quantità enorme di linguaggi di programmazione, i cui programmi vengono compilati in linguaggio assembly



Zoom sul C/C++



Conclusioni assumendo di fondere insieme i livelli 3 - 4



- Approfondiremo i vari livelli, focalizzandoci sui concetti più interessanti, seguendo il libro:
*"Architettura dei calcolatori. Un approccio strutturale",
Sesta Edizione. Andrew S. Tanenbaum, Todd Austin.
Pearson.*
- In parallelo, svilupperemo l'elaboratore su cui eseguire il nostro "Pong", seguendo il libro:
*"The Elements of Computing Systems".
Noam Nisan and Shimon Schocken. MIT Press.*
<http://www.nand2tetris.org>
Importante: iniziate a installare il software disponibile
(in particolare l'Hardware Simulator che useremo per realizzare il livello logico digitale e il CPU Emulator, quello che esegue il nostro "Pong" e gli altri programmi in linguaggio assembly)



Cosa mi aspetto da voi

- Che abbiate una buona idea degli argomenti spiegati nelle lezioni precedenti
 - L'architettura è a livelli, è molto difficile capire un livello senza aver capito il precedente
 - Se avete dubbi o domande chiedete chiarimenti all'inizio della lezione
- Che partecipiate: apprezzo
 - Chi si offre per correggere gli esercizi o partecipa al loro svolgimento con suggerimenti
 - Chi fa domande ragionevoli

Perchè affrontare questo viaggio insieme?

■ Motivi culturali

- L'architettura è alla base dell'esecuzione del software e quindi dell'informatica
- Non avrete molte occasioni di tornare su questi argomenti durante il corso di laurea

■ Motivi economici

- Come informatici dovrete riuscire a trovare lavoro ...
- ... ma le aziende non cercano studenti, cercano studenti bravi ...
- ... e non tutte le aziende sono uguali

- Esame finale: scritto 2 appelli a gennaio-febbraio, 3 a maggio-giugno-luglio, 1 a settembre
 - Possibile parte orale in caso le regole prevedano esame da remoto
- Consegne dei progetti relativi alla parte di laboratorio (lavoro individuale, facoltativo ma altamente consigliato)
 - E' possibile (ma più difficile) superare l'esame senza effettuare le consegne, ma non ottenere il massimo dei voti
 - Stabiliremo di volta in volta la deadline per il progetto
 - Alcune deadline cadranno durante il periodo delle lezioni, altre dopo il termine delle lezioni

Valutazione

- Esame scritto: 27 punti
- Progetto: 6 punti, divisi tra le varie consegne
 - I progetti vanno consegnati entro le scadenze
- 1 punto bonus per chi mostra la sua soluzione di un esercizio (e questa è ragionevolmente corretta)

■ Sul sito:

<https://virtuale.unibo.it/course/view.php?id=53548>

troverete tutto il materiale e gli strumenti per seguire al meglio il corso:

- slide usate durante le lezioni (aggiornate poco prima o poco dopo le lezioni)
- i file usati durante le esercitazioni in aula
- specifiche degli elaborati da sviluppare in autonomia
- lo strumento per effettuare la consegna dei vostri elaborati
- ...

- Lunedì: 13:00 - 15:00 G1 Via Zamboni, 67
Mercoledì: 15:00 - 17:00 G1 Via Zamboni, 67
Giovedì: 13:00 - 16:00 Aula magna Via Filippo Re, 10
- Divise in 2 moduli consecutivi:
 - Primo modulo tenuto da me
 - Secondo modulo tenuto dal Dott. Paolo Felli